

Accessories: the packet scan and the wrong-size gate — change notes

Date: 2026-09-27 · **Branch:** Hamthan-Dev **Migrations:** 1d4a32009101 (the squashed baseline, carries the table) · 20260928_garment_size_backfill (carries the data fix) **Revised 2026-09-28:** the alembic chain was squashed into one baseline the day after this work landed, which deleted the original 20260927_kit_packet_scan . See §10 for what moved where — no application code changed. **Touches:** store , materials (the style material spec), core rules, one new table

Who this is for. Anyone who did not write these changes — including the two frontend devs, whose store screen has a breaking change (§7). It assumes you know what a style, a SKU, a piece and a material lot are, and nothing else.

1. The problem, in one paragraph

A garment of size **L** was getting the **M**-size buttons. Nobody on the factory floor could see it, the client in Dubai could, and the bill was return freight plus a remade garment.

The system could not detect it **even in principle**. One scan on the garment (`part: "ACCESSORY"`) read the style's recipe and decremented *every* accessory line at once — four buttons, one zip, ten metres of thread — from a single tap. No physical packet was ever part of that exchange, so there was nothing whose size could be compared with the garment's.

There was also a second, quieter failure that produced the *opposite* symptom: garments shipping with **no zip at all**. §4 is about that one.

2. What the floor does now

Before — two scans, and the whole kit came off the recipe:

```
scan WORKER → scan GARMENT ("part": "ACCESSORY")
               ↳ decrements the button lot, the zip lot AND the thread lot
```

Now — three scans, and the third is the physical packet in the operator's hand:

```
scan WORKER → scan GARMENT → scan PACKET (LOT-ACC-000007)
                                   ↳ issues ONLY the line that packet matches
```

The operator repeats the third scan per packet: buttons, then zip, then thread. The garment is not complete until the last one.

Why three scans is not a step backwards. The store used to have a third scan — the *drawer* — and it was deleted on purpose, because it existed only to find a numbered box the system had invented. This third scan is the opposite: it is the label already printed on the packet, and it is the only thing in the building that can prove which size went in.

What happens when the size is wrong

The scan **fails**. A 409 , and *nothing* moves: no stock decremented, no ledger row, accessories_in untouched.

409 WRONG SIZE – NOT ISSUED. PC-222223 is a L garment and this packet is M; its recipe asks for L. Nothing has been taken from stock. Fetch the L packet, or ask a DM/MD to approve this substitution (request <id>) and scan it again.

header: X-Kit-Substitution-Request: 7f3a...

A DM or MD then approves or rejects it, and the operator **re-scans**. Until they do, nothing is issued.

This is the only place in the app that refuses work instead of recording it. Everywhere else — the skill gate, the stock shortfall — the work physically happened and losing the record would be worse, so it is logged with a warning. Here the wrong size is the failure. There is nothing worth preserving about it going in.

3. The approval flow

Step	Who	Endpoint	Effect
1	store operator	POST /store/scan with the wrong packet	409 , nothing spent, a PENDING request is written
2	DM / MD	GET /store/substitutions?status=PENDING	the queue, oldest first
3	DM / MD	POST /store/substitutions/{id}/approve	→ APPROVED . Issues nothing.
4	store operator	POST /store/scan with the same packet	issues it; request → CONSUMED

PENDING → APPROVED → CONSUMED , or PENDING → REJECTED .

Four decisions in that design are deliberate and worth knowing before you change anything:

a. The request row is committed *before* the 409 is raised. The scan has to fail, but the question has to outlive the failed request — otherwise the operator is told "wait for approval" and there is nothing, anywhere, for anyone to approve. Only that one row is written, and the garment has not been touched yet at that point in the scan, so the commit cannot leak a half-finished merge.

b. Approving is permission, not the issue. The operator is holding the packet; the DM is not. So the DM grants permission and the *operator's re-scan* writes the stock movement against the worker's own card. An approval that spent stock by itself would record a manager as having issued a packet they never touched — and whose hands the material passed through is the ledger's whole job.

c. It is idempotent on (piece, spec_line, lot) . The protocol is "scan, get refused, wait, scan again", so every re-scan must find the existing request rather than pile up identical asks for the DM to wade through. There is a DB unique constraint behind it.

d. Approve/reject is DM/MD only. The entire reason the scan is refused rather than flagged is that the person holding the wrong packet must not be the person who authorises it. A 403 for everyone else, including the Store Manager.

CONSUMED is terminal — one decision, one garment. But it is only set once the line owes **nothing**: a short issue (2 of 4 buttons) leaves the approval open so the rest can go in under the same decision. Getting that wrong was a bug I introduced and caught in review; there is a test named after it.

REJECTED can be approved later. "No" on Tuesday and "yes" on Wednesday is a real sequence, and forcing a second row for it would lose the first decision.

4. The second bug: garments shipping with no zip

This one is subtler and, over a whole order, more expensive.

The two size columns

A recipe line (`style_material_spec`) has **two** fields that look alike:

Column	Means	Compared with
<code>size</code>	the material's size — a 60 cm zip, an 18L button	<code>material_lot.size</code> , to find the stock
<code>garment_size</code>	which garments the line is for	<code>sku.size</code> , to decide if the line applies at all

`garment_size = NULL` means **every size**. That is what keeps one generic 18L button line at one row.

What was wrong

`garment_size` used to be **guessed** from `size` whenever `size` looked like a garment size — and *any bare number from 30 to 70* looked like one, because those are the EU jacket rungs.

So a DM entering a 60 cm zip as `size: "60"` got `garment_size = "60"` → mapped to **4XL** → the line applied to 4XL jackets **only**. A 45 cm zip went to XS.

And a line that reaches no garment is not a smaller recipe — it is no recipe. Follow it through:

```
the L/M/S/XL garments have no zip line
  → kit_required = False for them
  → piece_complete collapses to leather-and-lining
  → they are complete, sendable, shipped
  → with no zip, and no warning anywhere
```

From the garment's point of view the style simply declares no accessories. Nothing in the system had any reason to complain.

The fix, and the new hole it opened

`garment_size` is now **explicit only**. Nothing infers it.

- A material size that is **unmistakably** a garment size (`L` , `XXL`) with no `garment_size` is a **422** telling the DM to say which garments it is for.
- A **number** is never read as a garment size. `60` is centimetres until somebody says otherwise.
- Migration `20260928_garment_size_backfill` clears the rows the old guess already mis-scoped. Alpha ones are left alone — `L` beside `garment_size: "L"` is a line somebody meant, and clearing it would widen a genuinely size-specific button to every garment, which is the *opposite* error and that one spends stock.

But the guess was also the only thing stopping the reverse mistake: a DM entering `ZIP 48` , `ZIP 50` , `ZIP 52` — plainly three garment sizes — now has three **unscoped** lines, so every jacket gets all three zips. And `50` genuinely cannot be told from the token: it is a garment size on one client's sheet and a centimetre length on the next.

So that case is **asked**, not guessed — see blocker 5 below.

Two new release blockers

`release_blockers` had three; it now has five. A style cannot be released while:

4. **a size-varying accessory has no line for some size the order contains.** If *any* line for an article names a `garment_size` , that article is size-varying, so every ordered size must have its own line. Zip lines for M and L on an order that also runs S and XL is a blocker.
5. **several material sizes of one accessory, and none of them scoped** — the `ZIP 48/50/52` case.

Both are guarded against false positives, because a blocker that fires on the normal case teaches people the gate is noise:

- one unscoped line for the article covers every size and closes the question, so a generic 18L button never trips blocker 4;

- a per-colourway (SKU -scoped) line counts as covering its own SKU's size even when it names no `garment_size` — being that SKU's line already means that;
- one line per article is never ambiguous, however its size is labelled.

The gate runs at release because **that is the last moment anyone can be asked**. Before it the sheet is still a spreadsheet row; after it there are barcoded garments on the floor and the recipe is frozen and being spent.

5. Third fix: a named lot used to be spent unchecked

`issue_kit_nocommit` accepts a caller-supplied `material_lot_id` (for a substituted issue). It used to fetch that id and decrement it with **no comparison of any kind** — not article, not colour, not size, not even `is_active`. So the M-size button lot could be spent against an L garment's line and the ledger recorded it as correct.

The unpinned path had always matched on six columns; only the pinned one trusted the caller. `lot_fits_line` now checks it:

- **category and subtype are never waived** — a substitution is somebody saying "this size will do", never "a zip will do instead of a button";
- **article, colour and size are waived only for a DM-approved substitution**, since those are what a substitution actually substitutes.

A lot that does not fit comes back in `unresolved[]` with `reason: "MISMATCH"` and a sentence, rather than raising — one bad line must not lose the good ones.

6. Every file that changed, and why

New

File	What it is
<code>alembic/versions/20260928_1013_initial_migration.py</code>	the squashed baseline (<code>1d4a32009101</code>) — creates <code>kit_substitution_request</code> along with the whole schema
<code>alembic/versions/20260928_garment_size_backfill.py</code>	clears the mis-scoped <code>garment_size</code> rows. Split out of the original migration by the squash — see §10
<code>tests/integration/test_accessory_packet_scan.py</code>	20 tests: the refusal, the approval loop, the packet verdicts, the release gate
<code>tests/integration/test_garment_size_backfill.py</code>	9 tests for the migration's data fix
<code>docs/KAIROX_ACCESSORY_PACKET_SCAN_CHANGE_NOTES.md</code>	this document

`app/core/kit_rules.py` — the pure rules (no DB, no imports from `app.modules`)

Line	What
<code>size_matches</code> (:109)	the single size rule. <code>applies_to_size</code> , the release gate and the packet scan all delegate here, so <code>52</code> and <code>L</code> are one garment everywhere. A second spelling of it would be a second answer.
<code>accessory_size_gaps</code> (:133)	blocker 4

Line	What
<code>accessory_size_ambiguities (:179)</code>	blocker 5
<code>release_blockers (:218)</code>	two new optional kwargs; both default to <code>None</code> , so any caller not yet taught about sizes computes exactly what it always did

app/core/enums_barcode.py

- `KitSubstitutionStatus (:831)` — `PENDING / APPROVED / REJECTED / CONSUMED`
- `KIT_SUBSTITUTION_OPEN (:859)` — the two statuses still actionable
- three audit actions (:893-895). `...REQUESTED` is written by the scan that was **refused**, so the refusal itself is on the record — without it, the only trace of "somebody tried to put an M zip in an L jacket" would be the *absence* of a row, and a DM approving one would look like the first event in the story.

app/modules/barcode/models.py

`KitSubstitutionRequest (:586)`. Snapshots `garment_size`, `lot_size`, `article`, `colour`, `subtype` so the DM's queue needs no joins and still reads correctly after a lot is re-articled — the same reasoning as `piece_material_issue`'s four snapshot columns. `piece_id` and `spec_line_id` are **CASCADE**, not SET NULL: a pending approval for a deleted piece is not history, it is a question nobody can answer.

app/modules/materials/style_spec_service.py

Line	What	Why
:169	the <code>garment_size</code> 422	\$4
:273 <code>lot_fits_line</code>	validates a caller-named lot	\$5
:308 <code>match_packet</code>	the packet → recipe-line matcher	see below
:392 <code>_reads_as_garment_size</code>	narrowed to alpha rungs only	it is now only the 422's trigger; it infers nothing
:418 <code>applies_to_size</code>	now delegates to <code>kit_rules.size_matches</code>	one rule, three callers
:974-:1020	<code>_sku_facts</code> / <code>_coverage_dicts</code> / <code>size_checks</code>	feeds the two new blockers at all four <code>release_blockers</code> call sites
:1437 <code>issue_kit_nocommit</code>	hardened override + <code>substitution_approved</code>	\$5

`match_packet` matches in two tiers, and that split is the whole trick. Tier one ignores size entirely (subtype, article, colour, thickness), so the M packet still *finds* this style's BUTTON lines. Tier two then asks the two size questions separately: does this line belong on this garment, and is this the material the line asked for. Without the split, a wrong-size packet would simply fail to match and be reported as an unknown packet — the wrong message, sending the operator to hunt for a fault that is not there.

It returns six verdicts, and the store turns each into a different answer:

Verdict	HT TP	Meaning / the fix
<code>OK</code>	201	issue it
<code>WRONG_SIZE</code>	409	right article, wrong size → a DM decides

Verdict	HT TP	Meaning / the fix
NO_LINE_FOR_SIZE	409	the recipe has no line for this garment's size → a DM fixes the spec. Not a substitution to approve: every garment of that size is in the same state. This is blocker 4 showing up on the floor
NOT_IN_RECIPE	422	wrong packet entirely → check it, or record it off-spec with <code>POST /materials/issues</code>
NO_KIT	409	the style declares no accessories, or all its lines are for another colourway/size (<code>_no_kit_reason</code> says which of the three)
AMBIGUOUS	409	the packet matches several lines → combine or scope them

app/modules/store/service.py

Line	What
:203 <code>store_scan</code>	<code>lot_id</code> implies ACCESSORY; <code>part: ACCESSORY</code> without a packet is now a 422
:396 <code>_issue_packet</code>	loads the lot, matches it, handles the approval, issues one line, merges the checklist
:562 <code>_normalise_unresolved</code>	the read view calls the field <code>resolution</code> and the write path calls it <code>reason</code> ; every row now carries both, so the screen need not know which half it came from
:593 <code>_raise_for_substitution</code>	writes the request, commits, always raises
:660 <code>list_substitutions</code>	the queue. Ordered <code>created_at, id</code> — <code>id</code> is a tiebreak, not a sort: an unstable ORDER BY under limit/offset can serve one row on two pages and skip another
:718 <code>decide_substitution</code>	the DM's answer. Does not issue anything

app/modules/store/router.py / schemas.py

- `POST /store/scan` gains `lot_barcode` / `lot_id` , `qty` , `substitution_reason`
- three new routes (:101, :123, :141); `_APPROVERS` (:97) is DM/MD
- `lines[]` and `KitLineRequest` are removed — see §7

Docs kept in step

claude.md §8, §9, new §9a, §13, §14 · `docs/service/_src/{store,material,barcode}_system_guide.md` (and their regenerated `.docx`) · `docs/KAIROX_PRODUCTION-EVENT_SYSTEM_GUIDE.md` · `docs/PRODUCTION_FLOW_CURRENT.md` · `postman/store.postman_collection.json`

7. Breaking changes for the frontend

1. `POST /store/scan` with `"part": "ACCESSORY"` and no packet now returns 422. Send `lot_barcode` (or `lot_id`) instead. One request per packet.

```
// before – one call issued the whole kit
{ "employee_barcode": "EMP-000123", "piece_barcode": "PC-222223",
  "part": "ACCESSORY" }

// now – one call per packet, part omitted
{ "employee_barcode": "EMP-000123", "piece_barcode": "PC-222223",
  "lot_barcode": "LOT-ACC-000007" }
```

2. **lines[] on the scan request is gone.** The packet's own label decides which line is issued. If you were sending a partial quantity through `lines[].qty`, send top-level `qty` instead.
3. **The scan response gains substitution.** Non-null only when this scan spent a **DM-approved** wrong-size packet — show it, because a substitution that reads like a clean issue is a substitution nobody reviews. A *refused* one never reaches a 201.
4. **kit on the response is now the whole checklist**, not just the packet scanned — so the operator who has just done the zip can be told the buttons are still owed, while they are still standing at the terminal. `accessories_in` turns true only when every declared line is issued.
5. **Handle the 409 with the X-Kit-Substitution-Request header** as "held for approval", not as a generic error. The screen should show the request id and, for a DM/MD, a way to approve it.
6. **The recipe form must send garment_size explicitly** when a line is size-specific, and must surface the 422 when it forgets. There are two new release blockers to render — they are full sentences, meant to be shown verbatim.

8. How to verify it

```
# the new behaviour, end to end
pytest tests/integration/test_accessory_packet_scan.py -v      # 20 tests
pytest tests/integration/test_garment_size_backfill.py -v      # 9 tests

# nothing else broke
pytest tests/unit tests/integration -q                        # 1916 pass
pytest tests/functional tests/system tests/uat -q            # 1055 pass
```

Two failures in `tests/integration/test_procurement_*` are pre-existing and deliberate — the test's own docstring says it is expected to fail and that the failure is the bug report. No procurement code was touched here.

The migration has NOT been run against Postgres

It has only ever run against the test harness. The data fix lives in a module-level function

`clear_inferred_garment_sizes(bind)` rather than inline in `upgrade()` because it cannot be exercised by running the chain: since the squash the DDL does apply on SQLite (verified — both revisions run and the migration prints its count), but an alembic-built SQLite database rejects every INSERT, because the baseline sets `server_default=sa.text('now()')` on 138 timestamp columns and SQLite has no `now()`. Inline, the fix would have executed for the very first time on production.

Before applying it:

1. run it against a **scratch copy** of the database, not the live one;
2. read the line it prints — `cleared inferred garment_size on N accessory recipe line(s)` — and check `N` against what you expect;
3. run `compare_metadata` and confirm **zero drift**;
4. expect the release gate to start asking about those `N` lines. That is the point: they were silently mis-scoped, and now somebody has to say what they meant.

The `downgrade()` drops the table but **does not reverse the data fix**, deliberately. Re-deriving `garment_size` from a numeric size would restore the exact bug this removes — and it is not recoverable anyway: after the upgrade, a cleared line and a line a human deliberately left unscoped are the same row.

9. What was deliberately not changed

- **No new barcode type.** Accessories stay barcoded per `lot` (`LOT-ACC-000001`). One button out of a 5,000-button packet is any other button, so a per-unit code would mean printing 5,000 labels to learn nothing the packet count already says. A leather *hide* gets its own code because a hide is not interchangeable.
- **The breakdown importer still reads no accessory columns.** The recipe is entered by hand through the material-spec API and asked for at the release gate. That was already true and is unchanged.
- **Accessory reservations.** Shortfall is still *reported*, never reserved — nothing in the codebase can release a `MaterialReservation`, and `adjust_lot` / `retire_lot` hard-block on outstanding ones, so creating them here would wedge both permanently.
- **Leather and lining.** Still consumed at the cut, against the production event. Issuing them in the store would be a second, competing ledger for one material.

10. Revision 2026-09-28 — the chain was squashed under this work

The day after this landed, the alembic chain was squashed from ~45 revisions into a single baseline, `20260928_1013_initial_migration.py` (revision `1d4a32009101`, `down_revision = None`). That deleted the original `20260927_kit_packet_scan.py`.

No application code changed. Every file in `$6` outside `alembic/` and `tests/` is byte-identical to what shipped on 2026-09-27, and the full suite passes.

What the squash kept, and what it could not

A squashed baseline is autogenerated from the models, so it carries the **schema** and nothing else:

	Where it is now
<code>kit_substitution_request</code> — 20 columns, 5 FKs, <code>uq_kit_substitution_request</code> , 4 indexes	the baseline , verified column for column against the model
<code>style_material_spec.garment_size</code> + its index + its place in the unique constraint	the baseline
the <code>garment_size</code> data fix	nowhere — carried over by hand as <code>20260928_garment_size_backfill</code>

The data fix could not survive autogenerate, because a data migration is not part of a schema and a freshly-built database has no rows to correct. It matters only for a database that **already holds** mis-scoped rows — and since the original migration was never applied to Postgres, that is precisely the case that was still outstanding. It is applied unconditionally rather than after an audit: it is idempotent and a no-op on a clean database, so the alternative was deleting a real fix on the assumption that no database needs it.

To see how many rows it will touch before it touches them:


```
SELECT s.name AS style, m.article, m.subtype, m.size, m.garment_size
FROM style_material_spec m
JOIN style s ON s.id = m.style_id
WHERE m.category = 'ACCESSORY'
      AND m.garment_size IS NOT NULL
      AND m.size ~ '^[0-9]+$'
      AND upper(trim(m.size)) = upper(trim(m.garment_size));
```

Each row is a style whose **other** sizes are currently shipping without that accessory. The count should match the `cleared` inferred garment_size on N ... line the migration prints.

Two tests had to change, and one is not ours

Both loaded a migration **by filename**, so both died on a `FileNotFoundError` that said nothing about what had actually happened:

- `tests/integration/test_garment_size_backfill.py` (9 tests) now loads `20260928_garment_size_backfill.py`, and finds the baseline by its `down_revision = None` rather than by name — naming the file is what broke it, so it identifies a root revision by what it *is*. Its table check moved to asserting the **baseline** carries every column, rather than being deleted: that was the one thing the squash could plausibly have lost.
- `tests/unit/test_fk_delete_rules.py` (pre-existing, not part of this work) lost the `_NULLABLE_FKS` registry that `20260818_fk_setnull_all.py` carried. That registry's *job* is obsolete — it repointed constraints on an already-deployed database, and the baseline now creates every constraint with its rule inline — so invariant 3 asserts the property against the schema that actually ships: every nullable FK the models declare is created by the baseline with the **same** `onDelete`. Verified: 156/156 constraints parsed, 112 nullable FKs checked, zero missing, zero mismatches.

If you ever touch that parser: **match brackets, do not regex the call**. An autogenerated `ForeignKeyConstraint(...)` spans lines and nests brackets, so a regex reads about 70 of the 156 and reports the other 86 as missing their delete rule — 111 false failures. There is a second test, `test_the_baseline_is_parsed_at_all`, whose only job is to catch that happening again.